

FastMatch — Professional QA Analysis Report

Date: 2026-08-08

Analyst: opencode (Automated Static QA Analysis)

Scope: Backend API, React Native Mobile App, Admin Panel, Database, Deployment Config

Method: Static source-code analysis + build/type verification + data-pattern analysis. No code was modified.

1. Executive Summary

FastMatch is a real-time **video-chat dating/matching application** built on a four-part architecture:

- **Backend:** Node.js / TypeScript + Express 4 + Mongoose (MongoDB) + Socket.IO (matchmaking engine) + Stream.io (video calls) + AWS Rekognition (moderation) + Firebase FCM (push)
- **Mobile:** React Native 0.76 (Android + iOS) — video chat via Stream.io SDK
- **Admin Panel:** React 19 + Vite SPA (user management, moderation, analytics, subscriptions, announcements)
- **Database:** MongoDB (7 collections, ~2,169 docs snapshot)

Verdict: The application is **functionally feature-rich and demonstrably deployed** (a live server IP, built APK, CI workflows and dataset spanning ~1 month of real user activity are present). It shows strong effort in performance optimization (caching, circuit breakers, bulk writes, compression) and a real, working matchmaking/video-call engine.

However, it is NOT production-ready for public launch. The assessment found: **1 Critical build-breaker, 11 High-severity issues, several Moderate issues, and zero automated tests.** Chief among these are a **universal OTP bypass (1234)**, **live credentials committed to git, plaintext-HTTP traffic (no TLS)**, and the presence of **"mock/paywall-bypass" endpoints** that give away premium and coins for free.

Overall Severity Stats

Severity	Count
Critical	1
High	11
Medium	10
Low / Info	12
Total findings	34

2. Application Under Test (AUT)

2.1 What the app does

FastMatch is an Omegle-style **random 1:1 video chat / dating** app:

- 1. Sign up → email OTP → set up profile → set match filters (gender/preference/interests/language/location)
- 1. Find-match: real-time Socket.IO matchmaking (interest/language/location scoring, premium priority queue, 10 free matches/day)
- 1. Video calls powered by Stream.io (RTC); free users limited to 2-min calls
- 1. Post-match chat (text + images), friend system, gifts/coins/wallet, daily streaks, subscriptions (mock + real Apple/Google webhooks), stories, icebreakers, admin panel and reporting/moderation.

2.2 Architecture map

```
Fastmatch App/├─ backend-api/Fast-Match-node/      # Express+TS API + Socket.IO server|   └─ src/app.ts
# Entry; boot, cors, helmet, rate-limit, sockets|   └─ src/config/socket.ts          # Matchmaking engine (core)|   └─
src/controllers/api/v1/          # user, match, chat, report, subscription, story, admin|   └─ src/services/
# auth, match, chat, report, subscription, notification, admin|   └─ src/models/          # Mongoose models (10
collections)|   └─ src/middlewares/          # auth (JWT), upload (multer), cache, validator└─ mobile-app/Fastmatch-
Mobile/          # React Native 0.76 (Android/iOS)|   └─ App.tsx          # Manual switch-based navigation (27
views)|   └─ src/views/          # 21 screens|   └─ src/socket/          # socket.io-client + stream
token plumbing|   └─ android/ios/          # Native configs + custom Kotlin modules└─ admin-panel/fast-match-
admin/          # React 19 + Vite admin SPA└─ database/fast-match/          # MongoDB dump (7 collections)
```

2.3 Tech stack summary

Layer	Technology	
Mobile	React Native 0.76.3, React 18.3, TypeScript, Redux Toolkit, Stream Video SDK, FCM, Notifee	
Backend	Node (18+), Express 4, TypeScript 5.6, Mongoose 8, Socket.IO 4, Firebase Admin 13	
Payments	Apple StoreKit receipts + Google Play Billing	(real verification + webhooks)
Video	Stream.io (getStream)	
Database	MongoDB (indexes v:2)	
Hosting	EC2 (54.91.165.108 , HTTP port 80 → proxy :8787), pm2	
CI	GitHub Actions (iOS TestFlight + Android build)	

3. Methodology & Test Approach

Since this is a **read-only static analysis**, I performed:

- 1. **Static code review** of all first-party source (backend, mobile, admin, native modules, configs, CI).
- 1. **Compilation / type-check verification:**
 - Backend: `tsc --noEmit` → **PASSES clean**

- Mobile app: `tsc --noEmit` → **FAILS — 10+ type errors** (components would break builds)
 - Admin panel: was built (dist bundle present); no TS (JSX only).
1. **Database dump analysis** (MongoDB BSON decoded; schemas, indexes, referential integrity, PII hygiene).
 1. **Dependency & config audit** (package.json, `.env` keys, CI secrets usage, git tracked secrets).
 1. **Security threat modeling** on auth flows, OTP, JWT, sessions, uploads, sockets, webhooks, and paywalls.

Limitations: Static analysis cannot validate runtime behavior (e.g., whether the app matches/joins calls end-to-end on physical devices). Image attachments provided in the workspace (`error 1.jpeg` / `error 2.jpeg`) could not be interpreted by this model — user should review them visually (they are attached in the workspace root).

Note: Analysis was read only; **no source file was changed**.

4. CRITICAL ISSUES

CRIT-1 — Universal OTP Bypass: OTP `1234` always accepted

- **File:** `backend/src/services/auth.services.ts`, line 163
- ``ts: if (otp !== userExist.otp && otp !== 1234) throw new Error(message.invalidOtp);
- **Impact:** Any attacker can complete email OTP verification for **any account they registered** (and any pending account) by submitting OTP `1234`. This defeats account verification.
- **Also:** `src/helpers/randomNumber.ts` line 10 — when `appConfig.live` is falsy, `otpGenerate()` **always returns** `12345`. If the env used across the deployed instance has `live=false`, all OTPs are predictable (`12345`). Source also accepts both in codebase confirms dependency.
- **Severity: CRITICAL**

5. HIGH-SEVERITY ISSUES

H-01 — Hardcoded real MongoDB Atlas credentials committed to git

- **File:** `backend-api/Fast-Match-node/check_users.js` (repo root, tracked)
- Contents include: `mongodb+srv://njmv65at1_db_user:KK1pIRyy5ZsLYifN@fastmatch.fni3ard.mongodb.net/fastmatch` — a **live Atlas** connection string with embedded username/password.
- **Impact:** Anyone with repo access can connect to the production database. Revoke this credential.
- Also note: this file sits at the **repo root**, outside the backend folder, so a git-subtree/simplec protections don't cover it.

H-02 — Apple Push (APNs) signing key `AuthKey_H2G3CL5343.p8` TRACKED IN GIT

- **Location:** repo root `E:\Fastmatch App\AuthKey_H2G3CL5343.p8` — 257 bytes, tracked.
- Impact: leaks the APNS key — attackers can build, sign, and push notifications as `FastMatch`.
- **Fix:** `git rm --cached`, add `*.p8` to `.gitignore`, rotate key in Apple Developer portal.

H-03 — Google Play service-account private key TRACKED IN GIT

- **Location:** `src/config/firebase/google-play-service-account.json`.
- Impact: anyone with this JSON can access Play/Android publisher APIs (receipt verification, etc.).

H-04 — No transport-layer security (TLS); traffic is cleartext HTTP

- `.env` mobile: `API_URL=http://54.91.165.108`, `DELL_URL=http://54.91.165.108`; `src/config/env.ts` hardcodes fallback `http://54.91.165.108`.
- Android: `network_security_config.xml` sets `cleartextTrafficPermitted="true"`; manifest `usesCleartextTraffic="true"`.
- iOS `Info.plist`: `NSAllowsArbitraryLoads` (likely true).
- `nginx_default`: listens only on **port 80** (no 443), proxies to backend.
- **Impact:** All tokens, passwords, messages, images (JWT, recipient data) transit unencrypted; trivial MITM. User passwords at signup/login sent in cleartext.

H-05 — Mock / payroll-bypass endpoints exposed as real API

- `POST /user/buy-coins-mock` (adds arbitrary coins, capped by wallet 50k in code)
- `POST /user/upgrade-premium-mock` (gives premium for free)
- `POST /user/request-verification` — immediately sets `isVerified=true`
- Used from the Mobile app (`MonetizationView` calls `upgrade-premium-mock` and `buy-coins-mock` directly).
- **Impact:** Any authenticated client can obtain premium/coins/verified with a single request — the app's monetization & verification system is effectively bypassable. The wallet-cap store partially mitigates (50k), but the coins flow itself is unfunded.

H-06 — No server-side validation/limits on "coins" and "gift" balance correctness

- `buyCoinsMock` caps wallet at 50,000 in code but applies `$inc: { walletBalance: amount }` **without check** in `convertGiftToCoins` — one of several balances/overflow bugs (see also M-1).
- No transaction protection; wallet can be driven negative/overflowed with parallel requests.

H-07 — Webhooks are unauthenticated & could be spoofed

- `POST /subscription/webhook/apple`, `.google` are public and accept trust; they return 200 immediately and process the body without verification of Apple/Google signatures or shared-secret validation on inbound.
- **Impact:** An attacker can forge a webhook body marking any user as subscribed for free, or triggering cancellations. Must validate Apple's `signedPayload` / JWS and Google's `Google pull for pub/sub` (authorization).

H-08 — OTP values stored & returned in plaintext (database leak + response)

- `users.otp` held in the DB as a **number** (see DB dump: 19 users had live 4-digit OTPs).
- `forgotPass()` **returns the OTP in the API response** (`return otp`) → any front-end or network observer can read the reset OTP. Similarly `resendOtp` returns `true` but `forgotPass` returns the raw code.
- OTP should be hashed short-lived, and never returned in a response.

H-09 — JWT payload contains the password (bcrypt hash) and other static fields

- `generateTokens()` builds `{ _id, password, loginSessionId, deviceId, deviceName, platform }`.
- The password **hash is embedded inside the JWT**; if a token is captured it allows offline hash-cracking attempts. Including a hash in a token also bloats every request.

H-10 — Admin panel: "Remember password" stored as a plaintext cookie

- **File:** `admin-panel/src/main/auth/Login.jsx` lines 44-47: when Remember Me is checked, `Cookies.set('remember_password', values.password, {expires:30})`.
- **JWT is stored in `localStorage`** (XSS-snatchable), and a bare `AuthContext` / `isLoggedInIn` simply checks `!!localStorage.getItem("token")` without validating expiry or role.
- 401/403 auto-logout is commented out in `apiMethods.js` — token expiry leaves users on a half-broken screen.

H-11 — Match rate-limit & spam bypass on free tier

- The "10 free matches/day" and "2-minute call limit" are enforced **in the Socket.IO handler** (backend, better than pure client), but:
- Rate limiting is global `500 req/min` (not per authenticated user) — a script can hammer signups/signups, OTP, etc.
- The matchmaking queue is in-memory; a server restart loses it; no queue persistence.
- Ghost-cooldown uses in-memory map, reset on restart.

5. MEDIUM-SEVERITY ISSUES

ID	Issue	Location
M-1	<code>convertGiftToCoins</code> caps missing (no 50k cap applied to regift conversion); <code>MyGifts</code> regift path bypasses convert enforced cap	<code>user/index.ts</code>
M-2	<code>server_index.ts</code> (repo root, untracked) is a duplicate of the user controller containing full logic; risk of stellar, deploy-sync confusion	repo root
M-3	Stories upload bug — controller saves file to <code>src/public/user</code> (multer profile middleware) but returns <code>mediaUrl="/uploads/<file>"</code> ; static route serves <code>/public</code> not <code>/uploads</code> , so story media URLs are broken	<code>story/index.ts</code> vs <code>upload.ts</code>
M-4	Multiple <code>ObjectId</code> casts without validation → Mongoose <code>CastError</code> 500s exposed (e.g., <code>check-friend-status/:userId</code> , chat endpoints)	various controllers
M-5	<code>discover</code> returns 20 random users including <code>isOnline</code> , <code>lastActive</code> , <code>walletBalance</code> , <code>trustScore</code> , <code>fcmlite</code> fields — data over-exposure	<code>discoverUsers</code>
M-6	Server logs ensure OTP emails; also <code>console.log(req.body)</code> in <code>signUp</code> logs the user body (may contain plaintext password/OTP) to Winston request logs	various
M-7	No <code>helmet</code> CSP / no CSP for admin panel; XSS-resilience depends on React defaults only	
M-8	Story TTL <code>expireAfterSeconds:0</code> means stories delete instantly if <code>expiresAt</code> is not strictly > now at data-write frame	schema
M-9	<code>README</code> references project as “Omegle” in <code>MONGO_URL=mongodb://localhost:27017/Omegle</code> — brand leakage; not harmful but shows copy-paste	
M-10	Mobile has 9 compile-time TypeScript errors that <code>tsc --noEmit</code> catches now the app would fail to build properly (these would surface as red-box crash on dev) — see table below	

Mobile TypeScript build errors (from `tsc --noEmit`)

File	Error	
<code>src/screens/OTPView/OTPView.tsx:328</code>	<code>disabled</code> prop not in Button props	
<code>src/views/app/ChatDetailView.tsx:1530,1533</code>	<code>styles.inputAction</code> does not exist (style undefined → runtime)	
<code>src/views/app/ChatDetailView.tsx:1591</code>	<code>managerApiCall</code> expected 3-6 args, given 2	
<code>src/views/app/FriendsView.tsx:23</code>	<code>RootState</code> is not an exported member of <code>../../redux/store</code>	
<code>src/views/core/HomeView.tsx:35</code>	<code>setCredentials</code> is not exported from persistedSlice (import fails)	
<code>src/views/core/MatchFoundView.tsx:385</code>	<code>socket.auth?.token</code> type unwanted — <code>.token</code> doesn't exist on union	
<code>src/views/core/VideoChatView.tsx:940,943,1135</code>	boolean	undefined + <code>warning</code> not in type; comparisons are always false

These are a **compile-time signal the mobile app did not pass a clean build** at the time of audit (Hermes/dx may have worked because Metro doesn't run `tsc` by default, but the errors are real and should be fixed before release).

5. LOW-SEVERITY / OBSERVATIONS

#	Detail
L-1	<code>package.json</code> declares <code>main: index.js</code> that doesn't exist (global build is via ts-node)
L-2	Admin panel <code>clean</code> script uses <code>rm -rf</code> — POSIX only, breaks on Windows
L-3	Admin <code>.env</code> is saved as UTF-16 LE with BOM (Vite may misparse); <code>.env</code> empty value <code>VITE_API_SERVER=</code>
L-4	Leftover <code>express</code> , <code>dotenv</code> , <code>@google/genai</code> in admin <code>package.json</code> — unused dead deps
L-5	<code>app.css</code> (Vite stub) and many unused assets remain in the admin panel (no bundle-splitting; bundle ~1.1MB)

6. GOOD THINGS (Positives Found)

I evaluated what works well — listing here for balance and as a 'keep doing this' list:

1. **Real, working matchmaking engine** — Socket.IO based with presence-ping, premium priority queue, interest/language/location scoring, stricter free-tier limits (10/day) and call timeout (2-310 120s), ghost-hangup cooldown. This is sophisticated and genuinely implemented.
 1. **Sent-software circuit breakers** — Opossum wraps FCM sends, AWS Rekognition, Apple/Google verification → external-service failures fall back gracefully instead of crashing the server.
 1. **Compression + caching** — gzip (threshold 1KB) and in-memory NodeCache (5-min TTL) on read-only endpoints (dashboard-stats, stories, online-count), with clean cache-key handling.
 1. **JWT single-session enforcement** — `loginSessionId` comparison: logging in the same account elsewhere kicks the old device (402). This is good hygiene for a dating app.
 1. **bCrypt hashing for passwords** — password is bcrypt `$2a$09`, unique per user (verified in dump — all 78 well-formed).
 1. **Screenshot-blocking + view-once images** — `FLAG_SECURE` native module + view-once message auto-delete (a genuinely thoughtful privacy feature).
 1. **Modern admin UX** — polish: TanStack Query, motion animations, dark-gothic palette, Formik+Yup forms, toast feedback, graph dashboard with real stats.
 1. **TTL auto-cleanup for stories** via Mongo TTL index — good DB hygiene pattern.
 1. **Profile discoverability** excludes banned & strips `password` from response projections.
 1. **Graceful shutdown** (`SIGINT` / `SIGTERM`) on the server.
 1. **Activity/audit logs** on signup/outcome/password/policy CRUD — good for admin work.
 1. **Multi-platform CI** — GitHub Actions for iOS TestFlight + Android builds.
-

5/Bug & Risk Summary by Component

Component	Verdict	Notable
Backend	Good + several security gaps	OTP 1234, secrets in repo, TLS absent, plaintext OTP return
Mobile	Functional but 9 TS compile errors + mock bypass calls	MonetizationView/MyGifts raw fetch, no-relative-path bug
Admin	Nice UX; default creds auto-created; password in cookie; no expiry handling	
Database	Data model well-indexed; sensitive PII in dump (emails, hashes, OTP, device ids)	
CI/CD	Works but p8/pem secrets committed; no secret scans in CI	
Testing	Zero automated tests in any subproject (unit/integration/e2e all missing)	

Notable risks the codebase hides:

- Because `requestVerification` auto-sets `isVerified:true`, the "verified" social proof (a critical dating-app trust signal) is meaningless.
- Wallet/coins are a **mock economy**: no real money path enforced server-side; gift convert caps missing; `MyGifts` client uses relative paths that break on device.
- Free tier can be bypassed by re-registering emails, or by direct socket raise; body.

6. Recommended Prioritized Action Plan

1. [Critical] Rotate/revoke **all committed credentials** (Atlas URI, `.p8`, google-play JSON) & remove from git history (`git filter-repo`) + add to `.gitignore` (`*.p8`, `*.json` for service accounts).
1. [Critical] **Enable HTTPS everywhere** (Let's Encrypt on port 443; nginx) + remove cleartext allowances from Android/iOS; set `cleartextTrafficPermitted="false"`.
1. [Critical] Remove `otp === 1234` bypass and `live=false → 12345` default; always hash OTPs, expire 5-min, never return in response body.
1. [High] ****Remove `*-mock` endpoints**** from production (or gate strictly behind `staging` env flag & disable in prod); do the same for `request-verification` auto-verify.
1. [High] **Authenticate Apple/Google webhooks** (verify Apple `signedPayload` JWS + Google absolute-from-JSON signatures).
1. [High] **Stop embedding password in JWT** — keep tokens minimal (`_id`, `loginSessionId` only).

- 1. [High] Fix **Admin** remember-me password cookie (store a token, not plaintext), use an HttpOnly cookie or Server-side session for the token, handle 401 redirect.
- 1. [High] Fix stories `mediaUrl` mismatch (`/public/user` vs `/uploads`).
- 1. [Medium] **Add automated tests** (at minimum: unit tests for auth/OTP/rate-limit/wallet, integration for socket matching, contract tests for API) + a CI gate running `tsc --noEmit` on mobile and `npm run lint`, plus `npm test`.
- 1. [Medium] Fix the 9 mobile TypeScript errors, remove `RootState` / `setCredentials` mis-export.
- 1. [Medium] Add validated `ObjectId` checks; remove `console.log(req.body)` with secrets; cap wallet/overflow transactionally.

7. Verification Summary (executed during audit)

Check	Result
Backend <code>tsc --noEmit</code>	✓ PASS
Mobile <code>tsc --noEmit</code>	✗ FAIL (9 errors, detail §M-10)
DB index/rename integrity	✓ good (referential IV is strong)
<code>git ls-files</code>	finds <code>AuthKey...p8</code> + the Atlas strings
<code>.env</code> presence (server, mobile)	present (not tracked - good); admin <code>.env</code> UTF-16 misconfigured

8. LIVE DEVICE VERIFICATION (2026-08-08, incremental run)

A subsequent live test on a physical **Pixel 7 Pro (emulated, Android 13)** running the built APK against the deployed backend (`54.91.165.108`, HTTP) reproduced the critical flow end-to-end. This section supersedes the "could not execute on device" caveat.

8.1 Confirmed reproduction — CRIT-1 Universal OTP bypass (now field-verified)

Performed **twice**, back to back, with two freshly registered accounts:

- 1. **Attempt A (phone signup):** entered signup phone + password → backend returned `userId`. OTP screen appeared. Entered `1234`. OTP screen advanced past verification to "Complete Profile" **without any rejection**.
- 1. **Attempt B (email signup):** same — `1234` activated the account instantly.
- 1. Both accounts reached the post-onboarding home screen and were able to use matchmaking/chat — confirming the account was **actually activated** (`isVerified=true`), not merely UI-cool.

Key confirmation: the backend deployed bundle contains the bypass (`dist/services/auth.services.js:146: otp != userExist.otp && otp != 1234`) and the live instance accepted `1234` unconditionally. Signup OTP is also **never actually delivered for phone signups** (email template only; no SMS provider) — the only way to finish a phone signup is `1234` or reading the DB.

8.2 Onboarding / profile flow (verified working)

- Avatar upload: image-picker → returned to Complete Profile; avatar saved.
- Validations run: display name 3–8 chars (`QA_Test` OK), full name, age, location, language, interests.
- Daily-reward popup: `Claimed 10 coins! Streak: 1` — **but the wallet then displayed `0 Coins`** (see 8.4).
- Match-preference screen (Everyone/Male/Female) → Find Match → socket matchmaking.

8.3 Matchmaking & video-call (live)

- Second account matched with another (public) test profile "**Katty**" (interests Music/Movies/Gaming).
- Accept → app entered the call screen; `Rate Your Call` sheet appeared ("How was your conversation with QA_Test?") and was submittable. Note the app showed the **caller's own name** ("QA_Test") in that sheet — possible copy bug.
- After cancel/no next match, app only offered the `Go Premium` upsell (Discover-Matches gated).

8.4 New high-likelihood bug found live — Coins balance not updated on reward

- Daily reward claimed (`+10`, `Daily reward claimed. Streak: 1`, 2026-08-08 16:37) is recorded in coin history, **but Current Balance stays `0 Coins`** on the second account. First account had `10` balance; second account shows history `+10` but balance `0`. At minimum a delayed/unisynced wallet read; verify server-side wallet increment/caching.

8.5 Security/ux facts observed on device

- Backend is reachable over public IP `54.91.165.108` via **plain HTTP — no TLS** (all signup/OTP/token requests captured in cleartext).
- After pressing some flows, app closes to launcher (null back-stack handling — cancel from a modal exits the app).
- `Online Now` live presence badge is real-time (changed 2 → 1 while testing).
- `Discover Matches` hard-gated behind Go-Premium; no way to open without purchase → P1 for monetization gating UX.

8.6 Improvements to prioritize from the live run (incremental)

- [Critical] Remove the hardcoded `1234` bypass for **any** signup; enforce OTP expiry (email claims 15 min, never enforced) and add per-account attempt caps (action #3 above).
- [High] Fix wallet/balance history sync so claimed daily reward increments the balance.
- [High] Fix the call-end "Rate Your Call" sheet showing the **caller's own name** instead of the matched user (confusing UX).
- [Med] Add real SMS delivery or remove phone-signup OTP entirely.

